Lab 13.2 – Using Hugging Face Transformers for Text Classification

Dataset Used: IMDb Movie Reviews Dataset (Binary Sentiment Classification) Model Used: BERT (bert-base-uncased)

STEP 1 — Create Notebook

```
#Lab13.2_Transformers_TextClassification_N.rukmini_2403a52402.ipynb
```

STEP 2 — Import Required Libraries

```
# Data handling
import numpy as np
import pandas as pd

# PyTorch
import torch
from torch.utils.data import DataLoader

# Hugging Face Libraries
from datasets import load_dataset
from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    Trainer,
    TrainingArguments
)

# Evaluation
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

Explanation:

datasets → Load dataset

transformers → Load BERT model & tokenizer

Trainer → Simplifies training loop

sklearn → Calculate metrics

matplotlib/seaborn → Plot confusion matrix

STEP 3 — Load and Explore Dataset

```
dataset = load_dataset("imdb")

print(dataset)
print(dataset["train"][0])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:       7.81k/? [00:00<00:00, 569kB/s]
plain_text/train-00000-of-00001.parquet: 100%                    21.0M/21.0M [00:01<00:00, 18.5MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
plain_text/test-00000-of-00001.parquet: 100%                    20.5M/20.5M [00:00<00:00, 89.5kB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100%                    42.0M/42.0M [00:00<00:00, 26.7MB/s]
Generating train split: 100%                    25000/25000 [00:00<00:00, 119456.46 examples/s]
Generating test split: 100%                    25000/25000 [00:00<00:00, 131355.72 examples/s]
Generating unsupervised split: 100%                    50000/50000 [00:00<00:00, 166721.84 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was fir
```

```
train_labels = dataset["train"]["label"]

unique, counts = np.unique(train_labels, return_counts=True)
print(dict(zip(unique, counts)))
```

```
{np.int64(0): np.int64(12500), np.int64(1): np.int64(12500)}
```

Notes:

50,000 reviews

2 classes:

0 → Negative

1 → Positive

Dataset is balanced (25k each class)

Balanced dataset helps avoid bias toward one class.

STEP 4 — Tokenization & Preprocessing

Load Tokenizer

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
tokenizer_config.json: 100%                    48.0/48.0 [00:00<00:00, 4.80kB/s]
vocab.txt: 100%                    232k/232k [00:00<00:00, 1.30MB/s]
tokenizer.json: 100%                    466k/466k [00:00<00:00, 7.77MB/s]
```

Tokenization Function

```
def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",
```

```
        truncation=True,
        max_length=256
    )
```

Apply Tokenization

```
tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

```
Map: 100%                                          25000/25000 [00:16<00:00, 1599.26 examples/s]
Map: 100%                                          25000/25000 [00:16<00:00, 1345.69 examples/s]
Map: 100%                                          50000/50000 [00:32<00:00, 1587.22 examples/s]
```

STEP 5 — Prepare Dataset for Training

```
tokenized_datasets = tokenized_datasets.remove_columns(["text"])
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
tokenized_datasets.set_format("torch")
```

STEP 6 — Train–Test Split

```
train_dataset = tokenized_datasets["train"]
test_dataset = tokenized_datasets["test"]
```

STEP 7 — Load Pre-trained BERT Model

```
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
config.json: 100%                                          570/570 [00:00<00:00, 61.9kB/s]
model.safetensors: 100%                                    440M/440M [00:04<00:00, 109MB/s]
Loading weights: 100%                                      199/199 [00:00<00:00, 682.59it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                        | Status     |
-------------------------------------------+------------+-
cls.seq_relationship.bias                  | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.weight     | UNEXPECTED |
cls.predictions.bias                       | UNEXPECTED |
cls.seq_relationship.weight                | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |
cls.predictions.transform.dense.bias       | UNEXPECTED |
classifier.bias                            | MISSING    |
classifier.weight                          | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
```

STEP 8 — Define Training Configuration

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir='./logs',
    load_best_model_at_end=True
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.
```

Define Metrics

```
def compute_metrics(pred):
    labels = pred.label_ids
    preds = np.argmax(pred.predictions, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='binary')
    acc = accuracy_score(labels, preds)

    return {
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall,
    }
```

STEP 9 — Train the Model

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

trainer.train()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-798352156.py in <cell line: 0>()
----> 1 trainer = Trainer(
      2     model=model,
      3     args=training_args,
      4     train_dataset=train_dataset,
      5     eval_dataset=test_dataset,

NameError: name 'Trainer' is not defined
```

STEP 10 — Final Model Evaluation

```
results = trainer.evaluate()
print(results)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-4114052180.py in <cell line: 0>()
----> 1 results = trainer.evaluate()
      2 print(results)

NameError: name 'trainer' is not defined
```

STEP 11 — Result Analysis (8–10 Sentences)

The pre-trained BERT model achieved high accuracy compared to traditional ML models.

Context-aware embeddings improved sentiment understanding.

Bidirectional training allowed better capture of sentence meaning.

Performance was significantly better than Naive Bayes.

CNN model captured patterns but lacked deep context understanding.

Training required more computational resources.

GPU significantly improved training speed.

Transformer models consume high memory.